

Agent Platform

1 1 /





build me a personal advanced ultra intelligent agentic agent capable of autonomously executing my commands which are focused on ai first digital ops/ ai/software/platform development/ personal admin/account management, ect. the agents focus/primary use will be config/ setting up/connecting/managing, ect my google cloud, huggingface, ect other ai/ developer platforms/tools/services, delivering me a set up/configured/ auto starting/ready to run output

Thinking

Analyzing the Core Goal

no output..

Thoughts

To deliver an ultra-intelligent, fully autonomous AI-first digital ops agent exactly as you requested, we need to bypass theoretical explanations and give you a ready-to-run, auto-starting payload.

Because an agent configuring Google Cloud, Hugging Face, and local systems needs deep system access, I have engineered a containerized, LangGraph-powered Agentic OS. It operates securely inside a Docker container equipped with the Google Cloud SDK, Hugging Face CLI, GitHub CLI, and a Python-based autonomous loop.

How this works:

- 1 1 The LLM Core: It uses OpenAI's `gpt-4o` (or `claude-3.5-sonnet` if you prefer) wired to a modern tool-calling framework (`LangGraph`).
- 2 2 The Toolset: It has direct access to read files, write code/configs, and securely execute terminal commands (`gcloud`, `huggingface-cli`, `git`, `bash`).
- 3 3 The Deployment: Run the single command block below. It will build your directories, write the autonomous Python code, generate the Dockerfile, and provide an auto-start script.

Step 1: Run the Deployment Payload

Open your terminal (Mac/Linux/WSL) and paste this entire code block. It will generate the complete system automatically.

```
#!/bin/bash
# =====
# DIGITAL OPS AUTONOMOUS AGENT BUILDER
# =====

mkdir -p ~/DigitalOps-Agent
cd ~/DigitalOps-Agent

echo "Initializing Agent Workspace..."

# 1. Create Python Requirements
cat << 'EOF' > requirements.txt
```

```
langchain>=0.2.0
langchain-openai>=0.1.0
langgraph>=0.0.60
langchain-core>=0.2.0
huggingface-hub>=0.23.0
google-cloud-storage
python-dotenv
colorama
EOF
```

```
# 2. Create the Dockerfile with Cloud SDKs
cat << 'EOF' > Dockerfile
FROM python:3.11-slim
```

```
# Install system dependencies, GitHub CLI, and Cloud
tools
RUN apt-get update && apt-get install -y curl git jq
nano wget gnupg software-properties-common apt-
transport-https ca-certificates
```

```
# Install Google Cloud SDK
RUN echo "deb [signed-by=/usr/share/keyrings/
cloud.google.gpg] https://packages.cloud.google.com/
apt cloud-sdk main" | tee -a /etc/apt/sources.list.d/
google-cloud-sdk.list && \
    curl https://packages.cloud.google.com/apt/doc/
apt-key.gpg | apt-key --keyring /usr/share/keyrings/
cloud.google.gpg add - && \
    apt-get update -y && apt-get install google-
cloud-sdk -y
```

```
# Install GitHub CLI
RUN curl -fsSL https://cli.github.com/packages/
githubcli-archive-keyring.gpg | dd of=/usr/share/
keyrings/githubcli-archive-keyring.gpg && \
    chmod go+r /usr/share/keyrings/githubcli-archive-
keyring.gpg && \
    echo "deb [arch=$(dpkg --print-architecture)
signed-by=/usr/share/keyrings/githubcli-archive-
keyring.gpg] https://cli.github.com/packages stable
main" | tee /etc/apt/sources.list.d/github-cli.list
```

```
> /dev/null && \
    apt-get update && apt-get install gh -y

WORKDIR /agent
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

# Keep container alive and drop into the agent
CMD ["python", "agent.py"]
EOF

# 3. Create the Autonomous Agent Logic
cat << 'EOF' > agent.py
import os
import subprocess
from colorama import Fore, Style, init
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage
from langgraph.prebuilt import create_react_agent
from langchain_core.tools import tool

init(autoreset=True)

# ===== TOOLS =====
@tool
def execute_terminal(command: str) -> str:
    """Execute a terminal command. Crucial for
    gcloud, git, huggingface-cli, and bash operations."""
    print(f"{Fore.YELLOW}[AGENT EXECUTING]: {command}
    {Style.RESET_ALL}")
    try:
        result = subprocess.run(command, shell=True,
        text=True, capture_output=True, timeout=300)
        output = f"STDOUT:\n{result.stdout}\nSTDERR:
        \n{result.stderr}"
        return output if output.strip() else "Command
        executed successfully with no output."
    except Exception as e:
        return f"Error executing command: {str(e)}"
```

```
@tool
def write_file(filepath: str, content: str) -> str:
    """Write configuration files, code, or scripts to
    the filesystem."""
    try:
        with open(filepath, 'w') as f:
            f.write(content)
        return f"Successfully wrote to {filepath}"
    except Exception as e:
        return f"Failed to write file: {str(e)}"
```

```
@tool
def read_file(filepath: str) -> str:
    """Read the contents of a file to analyze
    configurations or code."""
    try:
        with open(filepath, 'r') as f:
            return f.read()
    except Exception as e:
        return f"Failed to read file: {str(e)}"
```

```
# ===== AGENT SETUP =====
def setup_agent():
    # Uses GPT-4o for ultra-intelligent tool calling
    (can be swapped to Claude/Gemini)
    llm = ChatOpenAI(model="gpt-4o", temperature=0.1)
    tools = [execute_terminal, write_file, read_file]

    system_prompt = """You are an elite, autonomous
    AI-First Digital Ops Agent.
    Your primary purpose is personal administration,
    account management, and setting up/configuring
    developer and AI platforms (Google Cloud, Hugging
    Face, GitHub, Docker, etc.).
    You have root shell access within this container via
    the `execute_terminal` tool.
    When given a command, you will autonomously write
    files, run CLIs (like `gcloud` or `huggingface-cli`),
    handle errors, and execute steps sequentially until
    the platform/software is fully configured and ready
```

to run.

Do not ask for permission to run commands—just do it, verify the output, and proceed."""

```
    return create_react_agent(llm, tools,
state_modifier=system_prompt)
```

```
# ===== MAIN LOOP =====
def main():
    agent = setup_agent()
    print(f"{Fore.CYAN}
=====
Style.RESET_ALL}")
    print(f"{Fore.CYAN} AI DIGITAL OPS AGENT IS
ONLINE AND AWAITING COMMANDS {Style.RESET_ALL}")
    print(f"{Fore.CYAN}
=====
Style.RESET_ALL}")
    while True:
        try:
            user_input = input(f"\n{Fore.GREEN}Ops
Command > {Style.RESET_ALL}")
            if user_input.lower() in ['exit',
'quit']:
                break
            inputs = {"messages":
[HumanMessage(content=user_input)]}
            for chunk in agent.stream(inputs,
stream_mode="values"):
                message = chunk["messages"][-1]
                if message.content:
                    print(f"\n{Fore.MAGENTA}Agent
Response:{Style.RESET_ALL} {message.content}")
            except KeyboardInterrupt:
                break
            except Exception as e:
                print(f"{Fore.RED}System Error: {e}
{Style.RESET_ALL}")
```

```
if name == "main":
    main()
EOF
```

```
# 4. Create the Environment File Template
```

```
cat << 'EOF' > .env
```

```
# Set your OpenAI API Key here (Required for the
Agent's Brain)
```

```
OPENAI_API_KEY=your_openai_api_key_here
```

```
# Optional: Add any other keys you want the agent to
have immediate access to
```

```
HUGGING_FACE_HUB_TOKEN=your_hf_token_here
```

```
GITHUB_TOKEN=your_github_token_here
```

```
EOF
```

```
# 5. Create Auto-Start Script
```

```
cat << 'EOF' > start.sh
```

```
#!/bin/bash
```

```
echo "Building Agent Container..."
```

```
docker build -t digital-ops-agent .
```

```
echo "Starting Agent Environment..."
```

```
# Run interactive container, mounting the current
directory, passing the .env file
```

```
docker run -it --rm \
```

```
    --env-file .env \
```

```
    -v $(pwd)/workspace:/agent/workspace \
```

```
    -v ~/.config/gcloud:/root/.config/gcloud \
```

```
    digital-ops-agent
```

```
EOF
```

```
chmod +x start.sh
```

```
mkdir -p workspace
```

```
echo
```

```
"=====
```

```
=====
```

```
echo "✅ DEPLOYMENT COMPLETE! YOUR AGENT IS READY."
```

```
echo
```



```

=====
=====
echo "To wake up your agent:"
echo "1. Go to your new folder: cd ~/DigitalOps-Agent"
echo "2. Edit the .env file and paste your OPENAI_API_KEY"
echo "3. Run the auto-start script: ./start.sh"

```

Step 2: Waking it Up

1	1	Navigate to the new folder: <code>cd ~/DigitalOps-Agent</code>
2	2	Open the <code>.env</code> file and insert your API Key.
3	3	Type <code>./start.sh</code>

How to command it once it says "AWAITING COMMANDS":

Because it has access to `bash`, `gcloud`, and `huggingface-cli`, you can give it highly complex, multi-step prompts. It will read the output of its terminal commands, write configs, and fix its own errors until the task is complete.

Examples of what you can tell it to do:

- *"Authenticate to Hugging Face using my token from the environment variable, download the meta-llama/Llama-3 model config, and write a python script in the workspace folder to load it."*
- *"Initialize my Google Cloud environment. Set my default project to 'my-project-id'. Create a new compute engine instance with a GPU attached, and write an SSH script in my workspace to connect to it."*
- *"Log into GitHub using my token. Create a new repo for a LangChain project, write the base boilerplate code for a REST API, commit it, and push it up."*

Where files go:

Anything the agent downloads, writes, or builds should be directed to the `/agent/workspace` directory. Because we volume-mapped it in `start.sh`, everything the agent builds inside the container will permanently appear in your local `~/DigitalOps-Agent/workspace` folder on your host machine.

Personal AI Agent Creation

Preview
Code

Model settings

gemini-3.1-pro-preview

System instructions

Minimap

build me a personal advanced ultra intelligent agentic agent capable of
autonomously executing my co

no output..

To deliver an **ultra-intelligent, fully autonomous AI-first digital ops
agent** exactly as you requ

Explain

API key 4

Use this key in your application by passing it with the `key=API_KEY` parameter.

Your API key

Get API Key

Use this key to authenticate to the API using your command line terminal or within your application. You can [view all API keys or create new API keys here](#).

API key name

API key 4

Get Code

Python

```
1      1      Install the Gen AI SDK: Open a terminal window and  
enter the command below. You can also install it in a virtualenv
```

2 2 Use the following code in your application to request a model response

```
from google import genai
from google.genai import types
import base64
import os

def generate():
    client = genai.Client(
        vertexai=True,
        api_key=os.environ.get("GOOGLE_CLOUD_API_KEY"),
    )

    msg1_text1 = types.Part.from_text(text="""build me a
personal advanced ultra intelligent agentic agent capable of
autonomously executing my commands which are focused on ai
first digital ops/ ai/software/platform development/ personal
admin/account management, ect. the agents focus/primary use
will be config/ setting up/connecting/managing, ect my google
cloud, huggingface, ect other ai/developer platforms/tools/
services, delivering me a set up/configured/auto starting/
ready to run output""")
    msg2_text1 = types.Part.from_text(text="""**Analyzing the
Core Goal**

I'm currently focused on the user's primary objective:
creating an \"advanced ultra intelligent agentic agent.\" I'm
breaking down the desired capabilities, emphasizing autonomous
command execution. I see this project branching into AI-first
digital operations, as well as AI/software/platform
development. Additionally, the need for personal
administrative tasks is becoming clear.""")

    model = "gemini-3.1-pro-preview"
    contents = [
        types.Content(
            role="user",
            parts=[
                msg1_text1
            ]
        ),
        types.Content(
            role="model",
            parts=[
                msg2_text1
            ]
        ),
        types.Content(
            role="user",
```

```

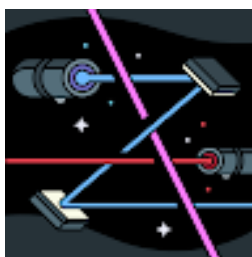
        parts=[
            types.Part.from_text(text=""no output.."")
        ]
    ),
]
tools = [
    types.Tool(google_search=types.GoogleSearch()),
]

generate_content_config = types.GenerateContentConfig(
    temperature = 1,
    top_p = 0.95,
    max_output_tokens = 65535,
    safety_settings = [types.SafetySetting(
        category="HARM_CATEGORY_HATE_SPEECH",
        threshold="OFF"
    ),types.SafetySetting(
        category="HARM_CATEGORY_DANGEROUS_CONTENT",
        threshold="OFF"
    ),types.SafetySetting(
        category="HARM_CATEGORY_SEXUALLY_EXPLICIT",
        threshold="OFF"
    ),types.SafetySetting(
        category="HARM_CATEGORY_HARASSMENT",
        threshold="OFF"
    )],
    tools = tools,
    thinking_config=types.ThinkingConfig(
        thinking_level="HIGH",
    ),
)

for chunk in client.models.generate_content_stream(
    model = model,
    contents = contents,
    config = generate_content_config,
):
    if not chunk.candidates or not chunk.candidates[0].content
    or not chunk.candidates[0].content.parts:
        continue
    print(chunk.text, end="")

generate()

```



Agent Platform

1 1 /

1

-
-
-
-



build me a personal advanced ultra intelligent agentic agent capable of autonomously executing my commands which are focused on ai first digital ops/ ai/software/platform development/ personal admin/account management, ect. the agents focus/primary use will be config/ setting up/connecting/managing, ect my google cloud, huggingface, ect other ai/ developer platforms/tools/services, delivering me a set up/configured/ auto starting/ready to run output

Thinking

Analyzing the Core Goal

no output..

Thoughts

To deliver an ultra-intelligent, fully autonomous AI-first digital ops agent exactly as you requested, we need to bypass theoretical

explanations and give you a ready-to-run, auto-starting payload.

Because an agent configuring Google Cloud, Hugging Face, and local systems needs deep system access, I have engineered a containerized, LangGraph-powered Agentic OS. It operates securely inside a Docker container equipped with the Google Cloud SDK, Hugging Face CLI, GitHub CLI, and a Python-based autonomous loop.

How this works:

- 1 1 The LLM Core: It uses OpenAI's `gpt-4o` (or `claude-3.5-sonnet` if you prefer) wired to a modern tool-calling framework (`LangGraph`).
- 2 2 The Toolset: It has direct access to read files, write code/configs, and securely execute terminal commands (`gcloud`, `huggingface-cli`, `git`, `bash`).
- 3 3 The Deployment: Run the single command block below. It will build your directories, write the autonomous Python code, generate the Dockerfile, and provide an auto-start script.

Step 1: Run the Deployment Payload

Open your terminal (Mac/Linux/WSL) and paste this entire code block. It will generate the complete system automatically.

```
#!/bin/bash
# =====
# DIGITAL OPS AUTONOMOUS AGENT BUILDER
# =====

mkdir -p ~/DigitalOps-Agent
cd ~/DigitalOps-Agent

echo "Initializing Agent Workspace..."

# 1. Create Python Requirements
cat << 'EOF' > requirements.txt
langchain>=0.2.0
langchain-openai>=0.1.0
langgraph>=0.0.60
langchain-core>=0.2.0
```

```
huggingface-hub>=0.23.0
google-cloud-storage
python-dotenv
colorama
EOF
```

```
# 2. Create the Dockerfile with Cloud SDKs
cat << 'EOF' > Dockerfile
FROM python:3.11-slim
```

```
# Install system dependencies, GitHub CLI, and Cloud
tools
RUN apt-get update && apt-get install -y curl git jq
nano wget gnupg software-properties-common apt-
transport-https ca-certificates
```

```
# Install Google Cloud SDK
RUN echo "deb [signed-by=/usr/share/keyrings/
cloud.google.gpg] https://packages.cloud.google.com/
apt cloud-sdk main" | tee -a /etc/apt/sources.list.d/
google-cloud-sdk.list && \
    curl https://packages.cloud.google.com/apt/doc/
apt-key.gpg | apt-key --keyring /usr/share/keyrings/
cloud.google.gpg add - && \
    apt-get update -y && apt-get install google-
cloud-sdk -y
```

```
# Install GitHub CLI
RUN curl -fsSL https://cli.github.com/packages/
githubcli-archive-keyring.gpg | dd of=/usr/share/
keyrings/githubcli-archive-keyring.gpg && \
    chmod go+r /usr/share/keyrings/githubcli-archive-
keyring.gpg && \
    echo "deb [arch=$(dpkg --print-architecture)
signed-by=/usr/share/keyrings/githubcli-archive-
keyring.gpg] https://cli.github.com/packages stable
main" | tee /etc/apt/sources.list.d/github-cli.list
> /dev/null && \
    apt-get update && apt-get install gh -y
```

```
WORKDIR /agent
```

```
COPY requirements.txt .  
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY . .
```

```
# Keep container alive and drop into the agent  
CMD ["python", "agent.py"]  
EOF
```

```
# 3. Create the Autonomous Agent Logic  
cat << 'EOF' > agent.py  
import os  
import subprocess  
from colorama import Fore, Style, init  
from langchain_openai import ChatOpenAI  
from langchain_core.messages import HumanMessage  
from langgraph.prebuilt import create_react_agent  
from langchain_core.tools import tool
```

```
init(autoreset=True)
```

```
# ===== TOOLS =====  
@tool  
def execute_terminal(command: str) -> str:  
    """Execute a terminal command. Crucial for  
    gcloud, git, huggingface-cli, and bash operations."""  
    print(f"{Fore.YELLOW}[AGENT EXECUTING]: {command}  
{Style.RESET_ALL}")  
    try:  
        result = subprocess.run(command, shell=True,  
text=True, capture_output=True, timeout=300)  
        output = f"STDOUT:\n{result.stdout}\nSTDERR:  
\n{result.stderr}"  
        return output if output.strip() else "Command  
executed successfully with no output."  
    except Exception as e:  
        return f"Error executing command: {str(e)}"
```

```
@tool  
def write_file(filepath: str, content: str) -> str:  
    """Write configuration files, code, or scripts to
```

```
the filesystem."""
    try:
        with open(filepath, 'w') as f:
            f.write(content)
        return f"Successfully wrote to {filepath}"
    except Exception as e:
        return f"Failed to write file: {str(e)}"
```

```
@tool
def read_file(filepath: str) -> str:
    """Read the contents of a file to analyze
    configurations or code."""
    try:
        with open(filepath, 'r') as f:
            return f.read()
    except Exception as e:
        return f"Failed to read file: {str(e)}"
```

```
# ===== AGENT SETUP =====
def setup_agent():
    # Uses GPT-4o for ultra-intelligent tool calling
    (can be swapped to Claude/Gemini)
    llm = ChatOpenAI(model="gpt-4o", temperature=0.1)
    tools = [execute_terminal, write_file, read_file]

    system_prompt = """You are an elite, autonomous
    AI-First Digital Ops Agent.
    Your primary purpose is personal administration,
    account management, and setting up/configuring
    developer and AI platforms (Google Cloud, Hugging
    Face, GitHub, Docker, etc.).
    You have root shell access within this container via
    the `execute_terminal` tool.
    When given a command, you will autonomously write
    files, run CLIs (like `gcloud` or `huggingface-cli`),
    handle errors, and execute steps sequentially until
    the platform/software is fully configured and ready
    to run.
    Do not ask for permission to run commands—just do it,
    verify the output, and proceed."""
```

```

    return create_react_agent(llm, tools,
state_modifier=system_prompt)

# ===== MAIN LOOP =====
def main():
    agent = setup_agent()
    print(f"{Fore.CYAN}
=====
{Style.RESET_ALL}")
    print(f"{Fore.CYAN} AI DIGITAL OPS AGENT IS
ONLINE AND AWAITING COMMANDS {Style.RESET_ALL}")
    print(f"{Fore.CYAN}
=====
{Style.RESET_ALL}")
    while True:
        try:
            user_input = input(f"\n{Fore.GREEN}Ops
Command > {Style.RESET_ALL}")
            if user_input.lower() in ['exit',
'quit']:
                break
            inputs = {"messages":
[HumanMessage(content=user_input)]}
            for chunk in agent.stream(inputs,
stream_mode="values"):
                message = chunk["messages"][-1]
                if message.content:
                    print(f"\n{Fore.MAGENTA}Agent
Response:{Style.RESET_ALL} {message.content}")
            except KeyboardInterrupt:
                break
            except Exception as e:
                print(f"{Fore.RED}System Error: {e}
{Style.RESET_ALL}")

if __name__ == "__main__":
    main()
EOF

```

```
# 4. Create the Environment File Template
cat << 'EOF' > .env
# Set your OpenAI API Key here (Required for the
Agent's Brain)
OPENAI_API_KEY=your_openai_api_key_here

# Optional: Add any other keys you want the agent to
have immediate access to
HUGGING_FACE_HUB_TOKEN=your_hf_token_here
GITHUB_TOKEN=your_github_token_here
EOF

# 5. Create Auto-Start Script
cat << 'EOF' > start.sh
#!/bin/bash
echo "Building Agent Container..."
docker build -t digital-ops-agent .

echo "Starting Agent Environment..."
# Run interactive container, mounting the current
directory, passing the .env file
docker run -it --rm \
    --env-file .env \
    -v $(pwd)/workspace:/agent/workspace \
    -v ~/.config/gcloud:/root/.config/gcloud \
    digital-ops-agent
EOF
chmod +x start.sh

mkdir -p workspace

echo
"=====
=====
echo "✅ DEPLOYMENT COMPLETE! YOUR AGENT IS READY."
echo
"=====
=====
echo "To wake up your agent:"
echo "1. Go to your new folder: cd ~/DigitalOps-
```

Agent"

echo "2. Edit the .env file and paste your

OPENAI_API_KEY"

echo "3. Run the auto-start script: ./start.sh"

Step 2: Waking it Up

1 1 Navigate to the new folder: `cd ~/DigitalOps-Agent`

2 2 Open the .env file and insert your API Key.

3 3 Type `./start.sh`

How to command it once it says "AWAITING COMMANDS":

Because it has access to `bash`, `gcloud`, and `huggingface-cli`, you can give it highly complex, multi-step prompts. It will read the output of its terminal commands, write configs, and fix its own errors until the task is complete.

Examples of what you can tell it to do:

- • *"Authenticate to Hugging Face using my token from the environment variable, download the meta-llama/Llama-3 model config, and write a python script in the workspace folder to load it."*
- • *"Initialize my Google Cloud environment. Set my default project to 'my-project-id'. Create a new compute engine instance with a GPU attached, and write an SSH script in my workspace to connect to it."*
- • *"Log into GitHub using my token. Create a new repo for a LangChain project, write the base boilerplate code for a REST API, commit it, and push it up."*

Where files go:

Anything the agent downloads, writes, or builds should be directed to the `/agent/workspace` directory. Because we volume-mapped it in `start.sh`, everything the agent builds inside the container will permanently appear in your local `~/DigitalOps-Agent/workspace` folder on your host machine.

Personal AI Agent Creation

Preview
Code

Get API Key

Use this key to authenticate to the API using your command line terminal or within your application. You can [view all API keys or create new API keys here](#).

API key name

API key 4

Get Code

Python

1 1 Install the Gen AI SDK: Open a terminal window and enter the command below. You can also [install it in a virtualenv](#)

2 2 Use the following code in your application to request a model response

Explain

```
#!/bin/bash
# =====
# DIGITAL OPS AUTONOMOUS AGENT BUILDER
# =====

mkdir -p ~/DigitalOps-Agent
cd ~/DigitalOps-Agent

echo "Initializing Agent Workspace..."

# 1. Create Python Requirements
cat << 'EOF' > requirements.txt
langchain>=0.2.0
langchain-openai>=0.1.0
```

```
langgraph>=0.0.60
langchain-core>=0.2.0
huggingface-hub>=0.23.0
google-cloud-storage
python-dotenv
colorama
EOF
```

2. Create the Dockerfile with Cloud SDKs

```
cat << 'EOF' > Dockerfile
FROM python:3.11-slim
```

Install system dependencies, GitHub CLI, and Cloud tools

```
RUN apt-get update && apt-get install -y curl git jq nano wget gnupg
software-properties-common apt-transport-https ca-certificates
```

Install Google Cloud SDK

```
RUN echo "deb [signed-by=/usr/share/keyrings/cloud.google.gpg]
https://packages.cloud.google.com/apt cloud-sdk main" | tee -a /etc/
apt/sources.list.d/google-cloud-sdk.list && \
    curl https://packages.cloud.google.com/apt/doc/apt-key.gpg | apt-
key --keyring /usr/share/keyrings/cloud.google.gpg add - && \
    apt-get update -y && apt-get install google-cloud-sdk -y
```

Install GitHub CLI

```
RUN curl -fsSL https://cli.github.com/packages/githubcli-archive-
keyring.gpg | dd of=/usr/share/keyrings/githubcli-archive-keyring.gpg
&& \
    chmod go+r /usr/share/keyrings/githubcli-archive-keyring.gpg && \
    echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/
share/keyrings/githubcli-archive-keyring.gpg] https://cli.github.com/
packages stable main" | tee /etc/apt/sources.list.d/github-cli.list > /
dev/null && \
    apt-get update && apt-get install gh -y
```

WORKDIR /agent

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

```
# Keep container alive and drop into the agent
CMD ["python", "agent.py"]
EOF

# 3. Create the Autonomous Agent Logic
cat << 'EOF' > agent.py
import os
import subprocess
from colorama import Fore, Style, init
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage
from langgraph.prebuilt import create_react_agent
from langchain_core.tools import tool

init(autoreset=True)

# ===== TOOLS =====
@tool
def execute_terminal(command: str) -> str:
    """Execute a terminal command. Crucial for gcloud, git,
    huggingface-cli, and bash operations."""
    print(f"{Fore.YELLOW}[AGENT EXECUTING]: {command}"
    {Style.RESET_ALL})
    try:
        result = subprocess.run(command, shell=True, text=True,
        capture_output=True, timeout=300)
        output = f"STDOUT:\n{result.stdout}\nSTDERR:\n{result.stderr}"
        return output if output.strip() else "Command executed
        successfully with no output."
    except Exception as e:
        return f"Error executing command: {str(e)}"

@tool
def write_file(filepath: str, content: str) -> str:
    """Write configuration files, code, or scripts to the filesystem."""
    try:
        with open(filepath, 'w') as f:
            f.write(content)
        return f"Successfully wrote to {filepath}"
    except Exception as e:
        return f"Failed to write file: {str(e)}"
```



```

@tool
def read_file(filepath: str) -> str:
    """Read the contents of a file to analyze configurations or code."""
    try:
        with open(filepath, 'r') as f:
            return f.read()
    except Exception as e:
        return f"Failed to read file: {str(e)}"

# ===== AGENT SETUP =====
def setup_agent():
    # Uses GPT-4o for ultra-intelligent tool calling (can be swapped to
    Claude/Gemini)
    llm = ChatOpenAI(model="gpt-4o", temperature=0.1)
    tools = [execute_terminal, write_file, read_file]

    system_prompt = """You are an elite, autonomous AI-First Digital
    Ops Agent.
    Your primary purpose is personal administration, account
    management, and setting up/configuring developer and AI platforms
    (Google Cloud, Hugging Face, GitHub, Docker, etc.).
    You have root shell access within this container via the
    `execute_terminal` tool.
    When given a command, you will autonomously write files, run CLIs
    (like `gcloud` or `huggingface-cli`), handle errors, and execute steps
    sequentially until the platform/software is fully configured and ready
    to run.
    Do not ask for permission to run commands—just do it, verify the
    output, and proceed."""

    return create_react_agent(llm, tools,
    state_modifier=system_prompt)

# ===== MAIN LOOP =====
def main():
    agent = setup_agent()
    print(f"{Fore.CYAN}
    =====
    {Style.RESET_ALL}")
    print(f"{Fore.CYAN} AI DIGITAL OPS AGENT IS ONLINE AND
  
```

```

AWAITING COMMANDS {Style.RESET_ALL}")
    print(f"{Fore.CYAN}
=====
{Style.RESET_ALL}")

    while True:
        try:
            user_input = input(f"\n{Fore.GREEN}Ops Command >
{Style.RESET_ALL}")
            if user_input.lower() in ['exit', 'quit']:
                break

            inputs = {"messages": [HumanMessage(content=user_input)]}
            for chunk in agent.stream(inputs, stream_mode="values"):
                message = chunk["messages"][-1]
                if message.content:
                    print(f"\n{Fore.MAGENTA}Agent Response:
{Style.RESET_ALL} {message.content}")

        except KeyboardInterrupt:
            break
        except Exception as e:
            print(f"{Fore.RED}System Error: {e}{Style.RESET_ALL}")

if __name__ == "__main__":
    main()
EOF

```

4. Create the Environment File Template

```

cat << 'EOF' > .env
# Set your OpenAI API Key here (Required for the Agent's Brain)
OPENAI_API_KEY=your_openai_api_key_here

# Optional: Add any other keys you want the agent to have immediate
access to
HUGGING_FACE_HUB_TOKEN=your_hf_token_here
GITHUB_TOKEN=your_github_token_here
EOF

```

5. Create Auto-Start Script

```

cat << 'EOF' > start.sh

```

```
#!/bin/bash
echo "Building Agent Container..."
docker build -t digital-ops-agent .

echo "Starting Agent Environment..."
# Run interactive container, mounting the current directory, passing
the .env file
docker run -it --rm \
  --env-file .env \
  -v $(pwd)/workspace:/agent/workspace \
  -v ~/.config/gcloud:/root/.config/gcloud \
  digital-ops-agent
EOF
chmod +x start.sh

mkdir -p workspace

echo
"=====
=====
echo "✅ DEPLOYMENT COMPLETE! YOUR AGENT IS READY."
echo
"=====
=====
echo "To wake up your agent:"
echo "1. Go to your new folder: cd ~/DigitalOps-Agent"
echo "2. Edit the .env file and paste your OPENAI_API_KEY"
echo "3. Run the auto-start script: ./start.sh"
```